

Pesquisa aprofundada sobre o modelo de supervisão

Como implementar, em um Docker-like system from scratch em Python, Cython e C++, o conjunto “processo pai do host + init interno; kill, wait, reap, shutdown limpo”

Recorte recomendado: Linux, cgroup v2, pidfd, signalfd, namespaces e supervisor nativo reanexável

Sumário

Resumo executivo	5
1. Escopo, premissas e posição de design	6
2. Definição dos conceitos centrais	7
2.1 Processo pai do host	7
2.2 Init interno	7
2.3 Kill	8
2.4 Wait	8
2.5 Reap	8
2.6 Shutdown limpo	9
3. Como o Docker original contextualiza o problema	9
3.1 O container como processo isolado, não como micro-VM	9
3.2 O papel do shim no Docker/containerd	10
3.3 Ordem de eventos e corridas de lifecycle	10
3.4 PID 1 , --init e os limites da aplicação como init improvisado	11
3.5 O que o Docker faz bem e o que um projeto novo pode fazer melhor	11
4. A recomendação arquitetural central para o projeto	11
5. System Design proposto	12
5.1 Macroarquitetura	12
5.2 Alocação de responsabilidades	13
5.3 Supervisor por container como MVP	14
5.4 Modelo de reinício e reattach	14
5.5 Modelo de estados	14
6. Low Level Design	15
6.1 O processo pai do host deve ser um reactor, não um emaranhado de threads	15
6.2 Estruturas de dados essenciais	15
6.3 Criação do processo do container	16
6.4 Injeção do init interno	16
6.5 Sinais e escopo de entrega	17
6.6 Wait imediato, reap imediato, cache permanente de ExitInfo	17

6.7 PR_SET_CHILD_SUBREAPER no host supervisor	18
6.8 PR_SET_PDEATHSIG como política opcional, não como pilar do design	19
6.9 signalfd , timerfd , epoll e nada de lógica em signal handlers	19
6.10 Cgroups como escopo de cleanup, não apenas de limitação de recursos	20
6.11 O init interno e o problema dos órfãos intra-container	20
6.12 I/O, PTY, FIFOs e propriedade do shim	20
6.13 exec como ampliação do mesmo modelo, não como exceção	21
6.14 Pseudocódigo do loop do supervisor	21
6.15 Pseudocódigo do init interno	22
7. Domain-Driven Design aplicado ao problema	22
7.1 Bounded contexts	22
7.2 Agregados e entidades	22
7.3 Value objects	23
7.4 Domain services	23
7.5 Eventos de domínio	23
8. Padrões de design aplicáveis	24
8.1 State	24
8.2 Strategy	24
8.3 Command	24
8.4 Observer / Publisher-Subscriber	24
8.5 Facade e Adapter	25
8.6 Builder	25
8.7 Template Method	25
9. Requisitos funcionais	25
9.1 Requisitos centrais	25
9.2 Requisitos funcionais derivados	27
10. Requisitos não funcionais	27
11. Cython como cola: o que fazer e o que evitar	28
12. Plano de implementação recomendado	29
Fase 1 — Provar o modelo sem namespaces	29
Fase 2 — Integrar namespaces, rootfs e cgroup v2	29
Fase 3 — Integrar o control plane Python	29

Fase 4 — Ordenação de eventos e observabilidade forte	30
Fase 5 — exec , corridas e recuperação pessimista	30
Fase 6 — Hardening e políticas avançadas	30
13. Estratégia de testes e cenários obrigatórios	30
14. Conclusão	31
Referências	32

Resumo executivo

Esta feature não é um detalhe periférico do runtime. Ela é a espinha dorsal operacional que separa um protótipo que “consegue iniciar um processo isolado” de um sistema de containers capaz de sobreviver a sinais, filhos órfãos, reinícios do daemon, corridas entre `start` e `exit`, e falhas de sincronização entre control plane e data plane. Em um Docker-like system construído do zero, a decisão arquitetural central é simples e dura: o processo real que supervisiona a workload do container não deve ser o processo Python do control plane. O pai do host precisa ser nativo, pequeno, previsível, reanexável e capaz de existir independentemente do ciclo de vida do interpretador Python.

No Docker contemporâneo, essa separação aparece na prática na combinação entre `dockerd`, `containerd`, `containerd-shim` e `runc`. O daemon gerencia metadados e a superfície operacional; o shim é o pai efetivo do processo do container, dono do I/O e da observação do `exit status`; o runtime OCI cria e inicia o ambiente; e, quando habilitado, um `init` mínimo dentro do container resolve o problema clássico de `PID 1`, reenvio de sinais e reaping de filhos. A documentação oficial do Docker e do containerd é explícita quanto a essa divisão de papéis.[R1][R2][R3][R7][R9][R10][R11]

Para o projeto proposto, a recomendação arquitetural é adotar um supervisor nativo em C++ por container — ou por sandbox, em um estágio posterior — e um `internal init` também nativo, idealmente um binário estático e mínimo injetado no bundle no momento de `create`. Python permanece como control plane, metadados, API e orquestração. Cython entra como fronteira tipada e eficiente entre o domínio Python e um cliente nativo pequeno, mas não como substituto de um shim ou como local de supervisão real. A razão é objetiva: `wait` pertence ao pai do processo, `pidfds` não sobrevivem a reinício de processo, e qualquer estratégia que dependa do processo Python como pai real do container compromete reattach, recuperação e limpeza correta.[R9][R10][R11][R13][R15][R22]

A implementação correta exige separar, conceitual e operacionalmente, quatro verbos que muita gente mistura. `Kill` é intenção de término, isto é, injeção de sinal ou escalonamento de parada. `Wait` é sincronização e observação do término, expondo `exit status` a clientes. `Reap` é o consumo kernel-level do estado do filho por `wait*`, removendo o zumbi da tabela de processos. `Shutdown limpo` é o protocolo que orquestra fechamento de `stdin`, sinal cooperativo, `grace period`, escalonamento, drenagem de descendentes, `flush` de I/O, persistência do estado final e `teardown` de recursos. Um runtime maduro não usa esses verbos como sinônimos; ele os encadeia com ordem, limites temporais e responsabilidades distintas.[R3][R9][R10][R12][R15][R19]

O recorte técnico mais saudável para um primeiro release não é “suportar qualquer Linux”, mas sim assumir Linux moderno com `cgroup v2`, `pidfd`, `waitid(P_PIDFD)` e `signalfd`. O ganho de robustez compensa a redução de compatibilidade. Sem isso, o projeto cai em um poço de fallback, PID reuse race, handlers frágeis e estados zumbis justamente na camada que mais precisa ser previsível. Em outras palavras: se há um lugar do runtime em que vale ser exigente com a baseline do kernel, é o modelo de supervisão.[R13][R14][R15][R16][R19][R22]

1. Escopo, premissas e posição de design

O tema desta pesquisa é o modelo de supervisão de processos em um sistema tipo Docker escrito do zero, usando Python para control plane e C++ para data plane, com Cython como cola. O recorte aqui assumido é Linux como alvo primário, preferencialmente em hosts com `cgroup v2` montado e kernel com suporte funcional a `pidfd`. Não se trata de uma limitação arbitrária. Trata-se de reconhecer que correção semântica de `kill`, `wait`, `reap` e `cleanup` é mais valiosa, no início do projeto, do que amplitude de compatibilidade.

Também parto de uma distinção importante entre duas ambições possíveis. A primeira é produzir um runtime de processos isolados “suficiente para demos”. A segunda é produzir uma base de runtime que possa sustentar futuro suporte a `exec`, restart policies, health checks, reattach, observabilidade e integração com um ecossistema mais amplo. O modelo de supervisão discutido aqui visa a segunda ambição. Por isso, algumas escolhas parecem mais caras no curto prazo — por exemplo, um shim nativo separado do processo Python — mas evitam refatorações estruturais dolorosas mais tarde.

A tese de design desta pesquisa é que o projeto deve tratar a supervisão como um bounded context próprio, com seu próprio modelo de estados, sua própria API local, sua própria estratégia de persistência transitória e sua própria lógica de recuperação. A API Python não deve “dar um `fork` e esperar”. Ela deve emitir comandos para um componente nativo que assuma a autoridade operacional sobre a árvore de processos do container. É essa separação que, no Docker real, permite que o daemon gerencie containers sem ser o pai efetivo de cada workload, e é essa separação que permitirá ao projeto sobreviver a reinícios do control plane, reanexar clientes e continuar produzindo semântica consistente de lifecycle.[R2][R9][R10][R11]

2. Definição dos conceitos centrais

2.1 Processo pai do host

O processo pai do host é o supervisor que vive no namespace do host e se torna pai do processo que inaugura o container. Em um design correto, ele não é um helper efêmero que dá `fork`, chama o runtime e desaparece. Ele é um processo persistente, pequeno, explicitamente dedicado a manter a relação parental com a workload, possuir os descritores relevantes de I/O, observar a saída, enviar sinais e servir uma interface local de `State`, `Kill`, `Wait`, `Delete`, `Shutdown` e, futuramente, `Exec`. No containerd, a descrição do shim é cristalina: o shim é lançado por container, é responsável por possuir o I/O dos processos do container, é o pai deles e permite reanexar I/O e receber o status de saída.[R10][R11]

Em termos conceituais, esse processo pai do host cumpre o papel de “âncora operacional” entre o mundo de metadados e o mundo de syscalls. Ele conhece o PID do init do container como visto do host, mantém os FDs de comunicação abertos, integra sinais e tempo em um loop previsível, e conserva a informação terminal necessária para que vários clientes lógicos façam `wait` sem depender do instante exato em que o processo morreu. Esse ponto é crucial: a responsabilidade do pai do host não termina em iniciar o container; ela começa ali.

2.2 Init interno

O init interno é um processo mínimo que roda como `PID 1` dentro do namespace de `PID` do container. Seu objetivo não é “gerenciar múltiplos serviços” no sentido de um `systemd` completo. Seu objetivo é resolver exatamente os problemas que a semântica especial de `PID 1` introduz: sinais com comportamento default não derrubam `PID 1` da forma que o usuário espera, filhos órfãos precisam ser reparentados e reaped, e `entrypoints` em shell ou scripts mal comportados frequentemente não propagam sinais aos descendentes. A documentação do Docker observa explicitamente que um processo rodando como `PID 1` em um container é tratado de forma especial pelo Linux e ignora sinais com ação default se não tiver código para tratá-los.[R4]

Por isso, um init interno bem desenhado precisa permanecer vivo como `PID 1`, iniciar a workload real como filho, observar `SIGCHLD`, repassar sinais conforme política e chamar `wait*` para reaped de descendentes. O Docker expõe isso na prática com `--init`, que injeta um init pequeno para reaping e gerenciamento básico do ciclo de vida; o Compose também oferece `init: true` com o mesmo propósito.[R7][R8] O projeto proposto deve tomar isso não como opção exótica, mas como baseline segura. Minha recomendação é que o init interno seja padrão e que o modo “sem init” exista apenas como compatibilidade avançada.

2.3 Kill

Kill é a operação de envio de um sinal ou de emissão de uma ação de término forçado contra a workload do container. No OCI runtime spec, a operação `kill <container-id> <signal>` deve enviar o sinal especificado ao processo do container.[R9] No Docker, `docker stop` usa uma semântica mais rica: primeiro envia `SIGTERM` ao processo principal e, após um `grace period`, envia `SIGKILL`; o primeiro sinal pode ser alterado por `STOPSIGNAL` no Dockerfile ou `--stop-signal` em `run/create`. [R3] `docker kill`, por sua vez, usa `SIGKILL` por padrão, mas pode enviar outro sinal.[R27]

Em um runtime próprio, `kill` não deve ser modelado como “matar o PID e pronto”. Ele deve carregar contexto: sinal, escopo do sinal, prazo de escalonamento, política de processo ou grupo e, idealmente, intenção semântica. Uma coisa é um “graceful stop” do lifecycle normal. Outra coisa é um “hard kill” por timeout, por policy externa ou por remoção forçada. Misturar essas intenções em uma única operação cega gera observabilidade ruim, estados inconsistentes e políticas impossíveis de auditar.

2.4 Wait

`Wait` é a operação pela qual um observador sincroniza com a terminação de um processo e obtém seu status final. Na superfície do containerd, isso aparece como um RPC `wait` com resposta de `exit_status` e `exited_at`. [R11] No plano kernel, isso acontece por `waitpid` ou `waitid`; em Linux moderno, `waitid` aceita `P_PIDFD`, permitindo esperar por um filho via process file descriptor.[R15] A distinção essencial é que `wait` no runtime não precisa coincidir um-para-um com uma chamada tardia ao kernel. O runtime pode — e deve — reaped o filho imediatamente quando ele sai e depois satisfazer chamadas lógicas de `wait` a partir de um `ExitInfo` cacheado.

Em outras palavras, `wait` é uma semântica para clientes; `reap` é uma obrigação com o kernel. Adiar o reaping esperando que algum usuário faça `wait` é um erro clássico. O sistema correto faz o contrário: reaped cedo para não acumular zombies e transforma o resultado terminal em um fato de domínio repetível.

2.5 Reap

`Reap` é a coleta efetiva do estado de término do filho por uma chamada `wait*`, removendo o zumbi da tabela de processos. O man page de `PR_SET_CHILD_SUBREAPER` é útil porque lembra que um subreaper cumpre o papel de `init` para descendentes órfãos e que, quando esses órfãos terminam, é o subreaper quem recebe o `SIGCHLD` e deve esperar por eles.[R12] Isso deixa claro que reaping não é detalhe de implementação; ele é parte do contrato funcional de qualquer processo que queira ser supervisor real.

No problema em questão, há dois domínios distintos de reaping. O primeiro é o domínio do host supervisor, que pode tornar-se subreaper para lidar com filhos órfãos no namespace do host, inclusive helpers de `exec` e processos de `setns`. O segundo é o domínio do init interno, que como `PID 1` no namespace de `PID` do container se torna o coletor natural dos órfãos intra-container. O `containerd` documenta explicitamente essa dupla responsabilidade: o shim age como subreaper para limpar containers e processos `setns`, mas, quando o container roda em um novo namespace de `PID`, o init dentro do container deve limpar processos órfãos antes de sair; além disso, há um detalhe importante sobre cross-namespace reparenting que impede supor que o subreaper do host resolverá tudo.[R10]

2.6 Shutdown limpo

Shutdown limpo é a orquestração ordenada de tudo o que foi definido acima. Ele começa antes do processo morrer e termina depois do processo já não existir mais. Envolve, no mínimo, fechamento opcional de `stdin`, entrega de sinal cooperativo, contagem de `grace period`, observação do `exit`, reaping imediato, possível escalonamento para `SIGKILL` ou `cgroup.kill`, espera pela drenagem do `cgroup`, flush de I/O, persistência do estado terminal e desmontagem de recursos. O Docker expõe apenas parte desse protocolo ao usuário final, mas internamente o modelo existe porque sem ele não há como garantir `stop`, `wait`, `rm -f`, `restart policies` nem ordem correta de eventos.[R3][R10]

Se eu tivesse de resumir a diferença entre um runtime principiante e um runtime maduro em uma frase, seria esta: o runtime principiante enxerga “o processo acabou”; o runtime maduro consegue provar, registrar e limpar as consequências desse término sem vazamento de estado.

3. Como o Docker original contextualiza o problema

3.1 O container como processo isolado, não como micro-VM

A documentação do Docker é direta ao afirmar que um container é, essencialmente, um processo isolado com seu próprio filesystem, sua própria rede e sua própria árvore de processos.[R1] Essa formulação é importante porque ela parece simples demais e, por isso mesmo, induz um erro de arquitetura comum: imaginar que um container runtime precisa apenas “começar um processo isolado”. Na prática, o que diferencia Docker de um simples `unshare + chroot` é a existência de um lifecycle model consistente, com estados, sinais, espera, reanexação de I/O, limpeza e recuperação.

O próprio `dockerd` é descrito como o processo persistente que gerencia containers. [R2] Mas ele não é, no desenho moderno, o pai direto de cada workload. Essa separação é precisamente o que torna o modelo recuperável. O daemon gerencia política, rede, imagem, storage, metadados e superfície de API; a ancoragem operacional de cada workload fica com a camada de runtime/shim.

3.2 O papel do shim no Docker/containerd

No ecossistema Moby/containerd, o shim é o componente que mais se aproxima do conceito de “processo pai do host” que interessa a esta pesquisa. O `shim.proto` do containerd afirma que o shim é responsável por possuir o I/O de cada container e processos adicionais, é o pai de cada container e permite reanexar I/O e receber o status de saída.[R11] Já o README de Runtime v2 detalha a arquitetura `shim + engine`: o shim é o executável realmente invocado pelo containerd, escuta um socket local e chama o runtime engine — por exemplo, `runc` — via `fork/exec` para criar, iniciar e parar o container.[R10]

O valor dessa arquitetura não é apenas separar responsabilidades; é separar tempos de vida. O shim pode continuar existindo mesmo que o daemon suba ou desça. Isso reduz acoplamento entre control plane e data plane e evita que “reiniciar a API” seja semanticamente equivalente a “perder o pai do processo do container”. Para um projeto em Python, essa lição é decisiva. O processo Python pode ser excelente como orquestrador, mas é um pai operacional ruim para workloads long-lived justamente porque seu tempo de vida pertence a outra classe de problemas.

3.3 Ordem de eventos e corridas de lifecycle

O README de Runtime v2 do containerd também traz uma lição mais sutil e muito valiosa: a ordem dos eventos de lifecycle importa. O documento exige que o shim publique `TaskStartEventTopic` antes de poder publicar `TaskExitEventTopic`, justamente para evitar corridas em que o processo sai tão rápido que um cliente observa `exit` antes mesmo de o `start` retornar.[R10] Essa é uma daquelas regras que parecem burocráticas até o dia em que um `docker exec`, um `wait` ou um restart policy entra em deadlock por causa de um evento fora de ordem.

Para o projeto proposto, isso significa que o modelo de supervisão não deve expor apenas syscalls embrulhadas. Ele precisa expor uma máquina de estados e uma política de publicação de eventos que respeitem causalidade. Em um sistema escrito do zero, errar essa ordem cedo costuma gerar uma coleção de correções locais que nunca atacam a causa. O correto é desenhar o estado e a ordem desde o começo.

3.4 `PID 1`, `--init` e os limites da aplicação como `init` improvisado

A documentação do Docker insiste em dois pontos que, juntos, explicam quase toda a motivação do `init` interno. Primeiro, `PID 1` em container é especial no Linux e não reage como o usuário médio espera a sinais `default`.^[R4] Segundo, `--init` insere um `init` pequeno que faz `reaping` e lida melhor com o `lifecycle` do processo.^[R7] A documentação de build do Docker acrescenta um terceiro ponto: usar a forma `exec` de `CMD` e `ENTRYPOINT` evita um `shell` pai que atrapalha o recebimento de sinais pelo executável principal.^{[R5][R6]}

Esses documentos expõem um fato desconfortável, mas útil: confiar que a aplicação do usuário, sozinha, será um bom `PID 1` é um ato de fé. Às vezes funciona; muitas vezes funciona só até a primeira carga real; e falha com frequência em `scripts shell`, `wrappers`, aplicações com filhos `demonizados` ou processos que nunca foram escritos para agir como `init`. Um runtime próprio não deve basear sua semântica de `shutdown` em esperança. Deve baseá-la em um `init` controlado pelo runtime.

3.5 O que o Docker faz bem e o que um projeto novo pode fazer melhor

O Docker privilegia compatibilidade e superfície de usuário estável, o que explica por que `--init` não é obrigatório e por que muito do comportamento de sinal ainda depende da imagem e da aplicação. Um projeto novo, porém, não precisa herdar essas concessões. Ele pode decidir que o modelo supervisionado é o padrão, que o `init` interno é sempre injetado, que `grace periods` são parte do contrato do container e que `wait` é sempre servido a partir de estado cacheado já `reaped`.

Esse é um ponto importante de postura de produto. Reproduzir o Docker não significa repetir cada decisão histórica do Docker; significa entender o problema que cada decisão tentou resolver e escolher, para o seu projeto, a variante com menor acoplamento e menor ambiguidade semântica.

4. A recomendação arquitetural central para o projeto

Minha recomendação para este projeto pode ser formulada em uma frase: Python deve ser o cérebro do sistema, não o pai real dos containers. A consequência prática dessa frase é a introdução de um supervisor nativo em C++, lançado por container, que persiste como processo separado, mantém a paternidade efetiva sobre o `init` do container e expõe uma API local de controle. Cython entra no processo Python apenas como uma ponte fina para lançar esse supervisor, conectar-se a ele e serializar chamadas de alta frequência ou baixa latência.

Há três razões técnicas fortes para isso. A primeira é semântica de `wait`: o processo que faz `wait*` corretamente é o pai — ou subreaper relevante — do filho. A segunda é durabilidade operacional: `pidfds` são descritores de processo vivos; eles não são um artefato que você persiste em um banco relacional e reabre depois de um restart do processo Python. A terceira é isolamento de falhas: reiniciar o interpretador, reciclar workers WSGI/ASGI, fazer deploy do control plane ou recarregar módulos Cython não deve romper a supervisão do processo do container.

Isso muda a maneira de usar Cython no desenho. Em muitos projetos, Cython é tratado como um atalho para “colocar C++ dentro do Python”. Aqui, isso seria insuficiente. O papel certo do Cython é encapsular uma fronteira estável: liberar GIL em chamadas bloqueantes, traduzir exceções nativas em exceções Python coerentes, marshalar estruturas tipadas e evitar overhead desnecessário. Mas a autoridade sobre `kill`, `wait`, `reap` e `cleanup` deve permanecer do lado do supervisor C++ e do init interno.

5. System Design proposto

5.1 Macroarquitetura

A macroarquitetura recomendada tem quatro peças. A primeira é o control plane em Python, responsável por API, CLI, persistência de metadados, desired state, policies, auditoria e reconciliação. A segunda é um adaptador Cython, mínimo e disciplinado, que serve de boundary layer entre o domínio Python e um cliente nativo local. A terceira é um supervisor C++ por container, ou por sandbox em uma fase posterior, rodando como processo do host e mantendo a paternidade real sobre o init do container. A quarta é um init interno C++, injetado no bundle do container e executado como `PID 1` dentro do namespace de PID.

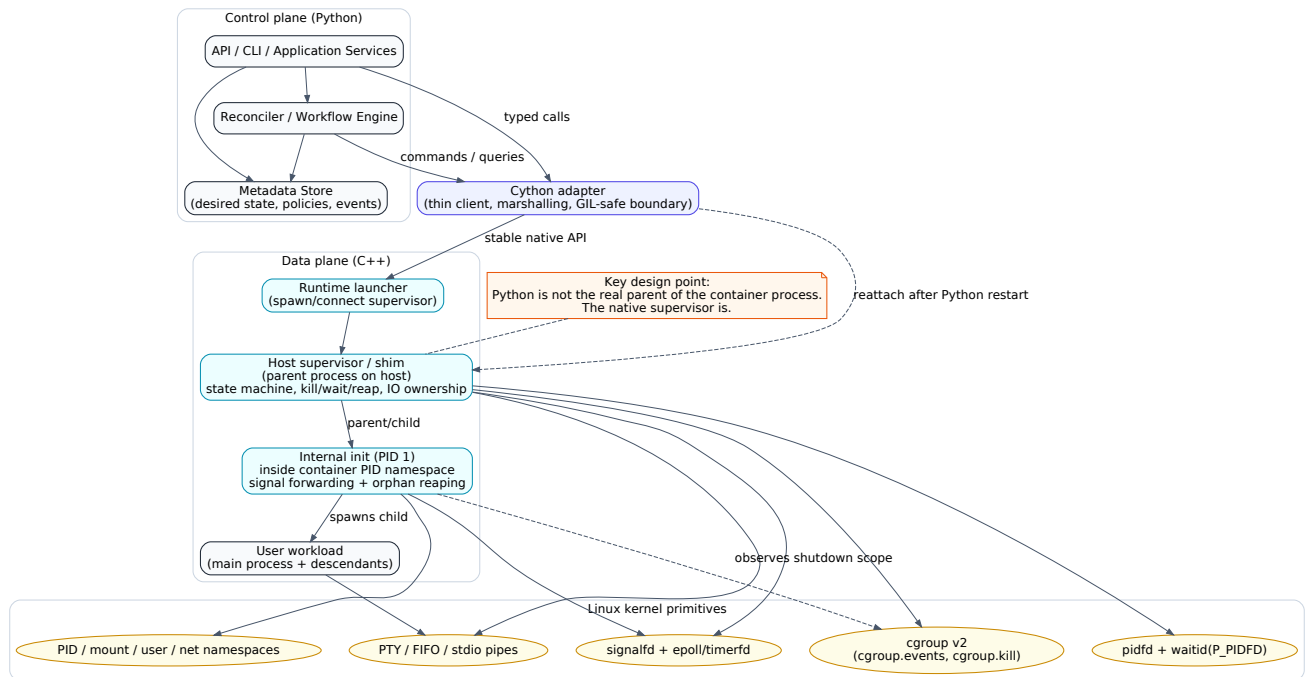


Figura 1. Separação entre control plane em Python, cola em Cython e supervisão efetiva em C++.

O control plane produz um `ContainerSpec` de alto nível, persistido em um metadata store. Esse spec contém imagem, rootfs resolvido, env, mounts, limites, política de parada, timeout, stop signal e configurações de namespaces/cgroups. O adaptador Cython converte esse spec em uma representação nativa e chama um launcher C++ que ou inicia um supervisor novo, ou se conecta ao supervisor existente daquele container. O supervisor, por sua vez, cria o ambiente, injeta o init interno no bundle, inicia o container e passa a servir chamadas como `State`, `Wait`, `Kill`, `Delete`, `CloseIO`, `Shutdown` e, futuramente, `Exec`.

5.2 Alocação de responsabilidades

O Python deve possuir tudo o que é política, fluxo de negócio e integração: autenticação, autorização, parsing de requests, tracking de desired state, agendamento de retries, apresentação de estados ao usuário, restart policies, contabilidade, mapeamento de IDs humanos para IDs de runtime e publicação de eventos de domínio externos. O C++ deve possuir tudo o que conversa com o kernel de forma intensa, sensível a latência ou semanticamente crítica: `fork/exec`, `clone3` quando usado, `pidfd`, `waitid`, `signalfd`, `epoll`, `cgroups`, `namespaces`, `stdio pipes` e o timing exato de `stop`.

Cython não deve se transformar em um segundo runtime nem em uma superfície enorme de wrapping C++. Quanto mais complexa for a API C++ exposta diretamente ao Cython, maior a fricção de manutenção, maior o risco de bugs de ownership e maior o custo de cada mudança interna do runtime. O ideal é expor a Cython um client API estreito e estável, preferencialmente via uma fachada C ou C++ muito simples, focada em requests e responses. Em outras palavras: Cython deve conhecer contratos, não internals.

5.3 Supervisor por container como MVP

Embora containerd permita um shim para muitos containers em alguns cenários, o desenho mais sensato para um primeiro release é um supervisor por container. Isso simplifica enormemente a causalidade dos eventos, o ownership de FDs, o `wait`, o mapeamento de logs, o diagnóstico e a recuperação. O custo é um processo extra por container no host, o que é perfeitamente aceitável em uma implementação inicial cujo objetivo principal é correção semântica.

Mais tarde, se o projeto evoluir para um conceito de sandbox ou pod, será possível agrupar múltiplos containers em um supervisor 1:N. Mas isso só deveria ser considerado depois que o modelo 1:1 estiver provado em cenários de corrida, parada forçada, reattach e `exec` concorrente. O erro clássico é otimizar a topologia do supervisor antes de acertar a semântica.

5.4 Modelo de reinício e reattach

Um dos ganhos mais importantes do supervisor externo é a possibilidade de reattach. Quando o processo Python reinicia, ele não precisa “reconstruir” o container a partir de PIDs soltos. Ele simplesmente escaneia um runtime directory local, por exemplo `/run/pycontainerd/<container-id>/`, descobre os sockets ativos dos supervisores e consulta o estado real. O supervisor continua segurando os `pidfds`, o estado terminal cacheado, os descritores de I/O e a memória de causalidade do lifecycle.

Se o control plane encontrar o bundle em disco, mas não conseguir conectar ao supervisor, então entra a trilha de recuperação pessimista: inspecionar o `cgroup`, checar se ainda há processos vivos, marcar o container como `UNKNOWN` ou `STOPPED` conforme o caso, e eventualmente disparar uma operação de `delete` offline semelhante ao `delete` binário descrito pelo containerd para limpar recursos quando a conexão RPC ao shim foi perdida.[R10] Essa distinção entre “reattach” e “garbage collection” é vital.

5.5 Modelo de estados

Recomendo separar estado desejado, estado observado e estado exposto ao usuário. O supervisor C++ deve trabalhar com um state machine operacional simples, inspirado no OCI: `creating`, `created`, `running`, `stopped`, com estados internos adicionais como `st`

`opping_gracefully`, `force_killing` e `deleting`. [R9] O Python, por sua vez, pode manter `desired states` como `start_requested`, `stop_requested`, `delete_requested` e um `read model` sintetizado mais amigável para a API.

Essa separação evita um problema comum: tentar usar um único campo de estado para representar tanto intenção quanto observação. A API diz “pare”, mas o supervisor ainda está em `running`; alguns segundos depois, o supervisor entra em `stopping`; depois expõe `stopped`; então o control plane decide se reinicia, remove ou mantém. Tudo isso é normal. Fica anormal quando a modelagem não faz distinção entre pedido e fato.

6. Low Level Design

6.1 O processo pai do host deve ser um reactor, não um emaranhado de threads

No nível de implementação, o supervisor por container deve ser predominantemente orientado a eventos, não a `thread-per-concern`. O coração do processo deve ser um reactor simples baseado em `epoll`, recebendo eventos de `signalfd`, `timerfd`, descritores de RPC local, PTY/FIFO e, quando útil, arquivos de `cgroup.events` passíveis de `poll`. O `man page` de `signalfd` destaca justamente a vantagem de tratar sinais como eventos em um file descriptor monitorável por `select/poll/epoll`. [R16]

Essa abordagem reduz races de memória compartilhada, evita handlers assíncronos inseguros e concentra as transições de estado em um único thread lógico. Pode haver threads auxiliares para cópia de I/O ou compressão de logs, mas a autoridade sobre o lifecycle do processo deve permanecer serializada. O resultado é menos throughput teórico por processo, porém muito mais previsibilidade — e previsibilidade é a moeda principal de um supervisor.

6.2 Estruturas de dados essenciais

No C++, o supervisor precisa de algumas estruturas centrais. `ContainerRecord` mantém identificação, socket local, `cgroup_path`, `bundle_path`, FDs de I/O, estado atual e políticas configuradas. `ProcessRecord` mantém `pid`, `pidfd`, `pgid` opcional, timestamps, flags de `running/stopped` e resultado terminal. `ExitInfo` representa código de saída, sinal de término, flag de core dump, instante monotônico e instante wall clock. `ShutdownPlan` representa sinal inicial, `grace period`, política de escalonamento e escopo do sinal.

No Python, o análogo deve ser mais semântico: `ContainerAggregate`, `LifecyclePolicy`, `SignalPolicy`, `GracePeriod`, `ContainerRuntimeView`. O control plane não precisa enxergar `pidfd` como conceito de domínio persistível; ele precisa enxergar identidade do runtime, estado observado e política aplicada. O `pidfd` é uma ferramenta interna do supervisor, não um fato de negócio.

6.3 Criação do processo do container

A criação do processo do container depende do restante do runtime, mas a supervisão impõe alguns requisitos claros. O supervisor precisa sair da fase `creating` apenas depois de existir uma relação parental concreta entre ele e o processo `init` do container. Idealmente, ele deve obter um `pidfd` do `init` o mais cedo possível, preferencialmente já na criação, para minimizar janelas de corrida. Em kernels modernos, isso pode ser feito por `clone3` com `CLONE_PIDFD`; em desenhos mais simples, um `fork/exec` seguido de `pidfd_open()` ainda é aceitável se a sequência de handshake for rigorosa.[R13]

A operação `create` também deve preparar o `bundle`, montar o `rootfs`, configurar `cgroups` e `namespaces` e reescrever o `process.args` para chamar o binário do `init` interno, passando a `workload` real como `payload`. O supervisor só publica `created` quando o ambiente estiver pronto e o `init` interno estiver plausivelmente instanciável. Só publica `running` após receber confirmação de `start` e cumprir a ordem causal de eventos descrita antes.[R9][R10]

6.4 Injeção do init interno

O `init` interno deve ser um binário pequeno, idealmente estático. A razão é prática: muitos containers modernos usam `scratch`, `distroless` ou imagens muito mínimas que não têm `dynamic loader` compatível com um binário injetado do `host`. Tini, por exemplo, mantém uma variante estática justamente por esse tipo de problema operacional. [R20] Para o projeto, a recomendação é gerar um `myrt-init` estático e montá-lo no `rootfs` do container em um caminho reservado, como `/.myrt/init`, executando-o como processo inicial do container.

Esse `init` não deve fazer mais do que o necessário. Ele bloqueia sinais relevantes, configura seu `event loop`, inicia a `workload` real como filho direto, mantém-se vivo como `PID 1`, repassa sinais conforme a política configurada e `reaped` todos os filhos por `wait*`. Não deve conter regras de negócio, não deve decidir `restart policies` e não deve se transformar em `process manager` genérico. Quanto mais estreito for seu contrato, mais confiável será seu comportamento.

6.5 Sinais e escopo de entrega

O desenho de sinais precisa ser explícito. Sinais cooperativos como `SIGTERM`, `SIGINT`, `SIGHUP` e um eventual `STOPSIGNAL` customizado devem entrar pelo supervisor do host e atingir o init interno. O init interno, então, decide se encaminha o sinal apenas ao filho direto ou ao grupo de processos da workload. Tini documenta que, por padrão, mata apenas o filho imediato, mas pode ser configurado para atingir o process group; dumb-init assume um comportamento de sessão/process group mais agressivo.[R20][R21]

Para um runtime novo, eu sugiro modelar isso como uma `SignalForwardingStrategy`. O modo de compatibilidade com Docker pode encaminhar apenas ao filho principal. O modo recomendado para produção pode usar process group, porque scripts shell e wrappers intermediários são muito comuns e, sem group signaling, descendentes podem sobreviver ao pai imediato. O importante não é qual política vence no primeiro release, e sim que a política seja uma escolha explícita do runtime, não um acidente do entrypoint.

6.6 Wait imediato, reap imediato, cache permanente de `ExitInfo`

Esta é a regra mais importante do LLD: o supervisor deve reaped o processo assim que observar sua saída, registrar o `ExitInfo` de forma imutável e satisfazer toda chamada posterior de `wait` a partir desse cache. O diagrama abaixo resume esse fluxo.

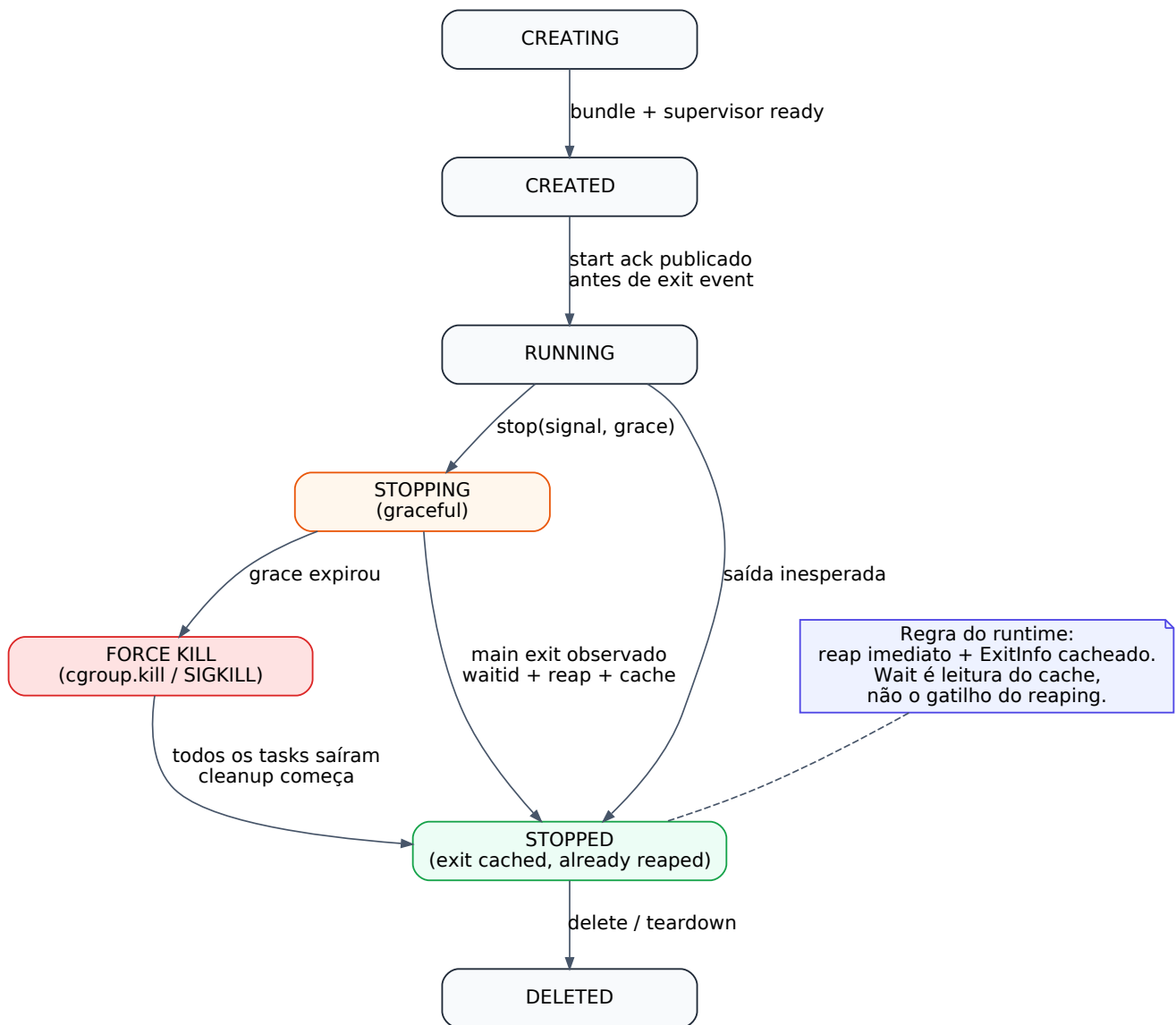


Figura 2. `Wait` é semântica de cliente; reaping é obrigação imediata do supervisor.

O man page de `waitid()` mostra o suporte a `P_PIDFD`, o que é excelente para o caso do processo `init` do container, porque permite esperar por um identificador não sujeito a PID reuse race.[R15] Já `pidfd_send_signal()` evita corridas semelhantes no envio de sinais.[R14] A combinação dos dois entrega exatamente a robustez que esse tipo de runtime precisa: referência estável para terminar, referência estável para observar. Uma vez consumido o status, `ExitInfo` deve se tornar parte do estado terminal do supervisor e nunca mais ser recalculado.

6.7 PR_SET_CHILD_SUBREAPER no host supervisor

O host supervisor deve chamar `prctl(PR_SET_CHILD_SUBREAPER, 1)` logo no início de sua vida. O manual do Linux explica que um subreaper cumpre o papel de `init` para processos descendentes órfãos e ressalta um detalhe importante: essa configuração não é herdada por filhos criados por `fork/clone`, mas é preservada através de `execve`. [R12]

Isso significa que o processo que pretende ser o coletor real de órfãos precisa setar essa flag em si mesmo; não adianta presumir que helpers ou workers a herdarão automaticamente.

Essa medida é especialmente útil para processos auxiliares de `exec`, `setns` e outras operações em que podem surgir descendentes fora da árvore “bonita” do `init` do container. O próprio `containerd` documenta o shim como subreaper e, ao mesmo tempo, reconhece que o `init` do container ainda precisa assumir parte da limpeza quando há novo namespace de PID e cenários de `setns`. [R10] Em resumo: o host supervisor deve ser subreaper, mas não deve fantasiar que isso elimina a necessidade do `init` interno.

6.8 `PR_SET_PDEATHSIG` como política opcional, não como pilar do design

`PR_SET_PDEATHSIG` permite pedir que um processo receba um sinal quando seu pai morrer, inclusive em cenários envolvendo subreapers reparenting ao longo da cadeia. [R17] Isso é útil para alguns modelos fail-stop, nos quais você prefere matar a workload se o supervisor nativo desaparecer de forma inesperada. Contudo, eu não faria dessa técnica o pilar da arquitetura. Um runtime Docker-like quer, em geral, sobreviver a reinícios do control plane e preservar a workload. Se o supervisor for o componente estável e reanexável, a política padrão pode ser “continuar vivo e permitir reattach”.

Ainda assim, vale manter `PDEATHSIG` como estratégia configurável. Em ambientes altamente controlados, pode ser desejável que a morte do supervisor implique a morte do container, especialmente quando a rastreabilidade operacional é mais importante do que disponibilidade. Isso cabe perfeitamente em uma `ShutdownPolicy` parametrizável.

6.9 `signalfd`, `timerfd`, `epoll` e nada de lógica em signal handlers

A recomendação de implementação para o supervisor e para o `init` interno é bloquear sinais relevantes com `sigprocmask`, recebê-los por `signalfd` e tratar tudo em um loop com `epoll`. Isso vale para `SIGCHLD`, `SIGTERM`, `SIGINT`, `SIGHUP` e quaisquer sinais de controle que o runtime quisesse interceptar. O `timerfd` complementa esse desenho ao representar o `grace period` como um evento de tempo explícito, evitando `sleep` ou `threads` temporizadoras espalhadas.

Esse modelo entrega três vantagens práticas. Primeiro, evita código em contextos `async-signal-safe`, onde quase nada interessante pode ser feito com segurança. Segundo, serializa causalidade: sinal recebido, estado atualizado, ação emitida, próximo estado. Terceiro, facilita testes, porque o lifecycle inteiro vira uma sequência de eventos de FD observáveis.

6.10 Cgroups como escopo de cleanup, não apenas de limitação de recursos

Muita gente lembra de cgroups apenas como mecanismo de limite de CPU e memória. Para supervisão, eles servem também como escopo confiável de cleanup. A documentação de cgroup v2 diz que `cgroup.events` expõe um campo `populated`, com notificação via `poll` quando o sub-tree deixa de ter processos vivos, e que escrever `1` em `cgroup.kill` mata todos os processos do cgroup e descendentes via `SIGKILL`, lidando apropriadamente com forks concorrentes e protegido contra migrações.[R19]

Isso é poderoso. Em um `force kill`, o runtime não precisa ficar iterando `cgroup.procs` e perseguindo PIDs em uma árvore que pode mudar enquanto ele a percorre. Ele pode usar `cgroup.kill` como último recurso forte quando disponível. Depois, pode observar `cgroup.events.populated` para saber quando o sub-tree realmente esvaziou antes de desmontar o rootfs e remover diretórios. Essa é uma das razões para assumir `cgroup v2` no recorte inicial: a semântica fica objetivamente melhor.

6.11 O init interno e o problema dos órfãos intra-container

O PID 1 de um namespace de PID é o destino natural de órfãos naquele namespace. O man page de `pid_namespaces(7)` lembra que filhos órfãos são reparentados ao `init` do namespace do pai, exceto quando um subreaper mais próximo está configurado.[R18] No contexto desta feature, isso significa que o init interno precisa manter um loop robusto de `waitpid(-1, WNOHANG)` ou `waitid(P_ALL, ...)` após `SIGCHLD`, drenando todos os filhos mortos, não apenas o processo principal da workload.

O containerd chama atenção para um detalhe ainda mais traiçoeiro: em cenários com `setns`, o reparenting cross-namespace não funciona do jeito intuitivo, então o init do container precisa limpar órfãos gerados por processos `exec` dentro do novo namespace de PID.[R10] Essa observação sozinha justifica desenhar o init interno como componente de primeira classe e não como um simples `exec` do programa do usuário.

6.12 I/O, PTY, FIFOs e propriedade do shim

O shim deve ser dono do I/O do container. O `shim.proto` do containerd explicita isso e expõe inclusive RPCs como `CloseIO` e `ResizePty`. [R11] A implicação para o projeto é direta: os endpoints de `stdout`, `stderr` e `stdin` não devem ser possuídos pelo processo Python que recebeu a requisição de `run`. Eles devem estar sob controle do supervisor, o que permite reattach de clientes, drenagem após saída do processo e recuperação mais limpa em cenários de restart do control plane.

Na prática, isso significa que o supervisor precisa criar e manter FIFOs ou PTY master/slave, encaminhar logs ao backend configurado e expor handles reanexáveis. O control plane Python pode abrir sessões e multiplexar clientes, mas não deve ser o único dono dos FDs. O dono canônico é o supervisor nativo.

6.13 `exec` como ampliação do mesmo modelo, não como exceção

Embora o foco desta pesquisa seja o `init` principal do container, o design deve ser compatível com processos adicionais iniciados por `exec`. Cada `exec` precisa de seu próprio `ProcessRecord`, sua própria semântica de `wait` e uma política clara de `kill`. O supervisor deve continuar sendo a autoridade sobre esses processos adicionais. O modelo do `containerd`, com RPCs separados de `Exec`, `Kill`, `Wait` e `Delete`, é uma boa referência operacional.[R11]

A melhor maneira de evitar bugs de `exec` é não tratá-lo como “uma chamada especial que contorna o lifecycle principal”. Ele deve reaproveitar o mesmo pipeline de observação, cache de `ExitInfo` e publicação ordenada de eventos. O que muda é só o escopo do `ProcessRecord`, não a teoria da supervisão.

6.14 Pseudocódigo do loop do supervisor

```
block_signals(SIGCHLD, SIGTERM, SIGINT, SIGHUP)
sfd = signalfd(mask)
tfd = timerfd_create()
ep = epoll_create()
epoll_add(ep, rpc_socket)
epoll_add(ep, sfd)
epoll_add(ep, tfd)
epoll_add(ep, cgroup_events_fd_if_used)

state = CREATED
while true:
    for ev in epoll_wait(ep):
        if ev == rpc_socket:
            cmd = recv_rpc()
            handle_command(cmd)
        elif ev == sfd:
            sig = read_signal()
            if sig == SIGCHLD:
                reap_all_children_and_cache_exitinfo()
            else:
                apply_supervisor_signal_policy(sig)
        elif ev == tfd:
            escalate_shutdown_if_needed()
        elif ev == cgroup_events_fd:
            if cgroup_populated() == 0 and state in {STOPPED, DELETING}:
                finish_cleanup()
```

O ponto importante desse pseudocódigo não é a sintaxe, mas a disciplina. O supervisor não faz lógica crítica em callbacks arbitrários. Ele converge tudo para um único loop de eventos e mantém sua máquina de estados em um lugar só.

6.15 Pseudocódigo do init interno

```

block_signals(SIGCHLD, SIGTERM, SIGINT, SIGHUP, SIGQUIT)
sfd = signalfd(mask)
child = spawn_user_process(argv, env, signal_forwarding_strategy)

while true:
    sig = read(sfd)
    if sig == SIGCHLD:
        drain_wait_nonblocking()
        if main_child_exited:
            if remaining_descendants and graceful_drain_enabled:
                signal_remaining_descendants()
                continue_until_reaped_or_deadline()
            exit_with_main_child_status()
        else:
            forward_signal_to_child_or_group(sig)

```

O que faz esse processo ser “init” não é volume de código; é o fato de permanecer PID 1, centralizar SIGCHLD, possuir política de forwarding e nunca delegar reaping à aplicação do usuário.

7. Domain-Driven Design aplicado ao problema

7.1 Bounded contexts

Vejo quatro bounded contexts principais relacionados a esta feature. O primeiro é `Container Catalog`, no qual vivem identidade do container, desired state, labels, meta-data operacional e persistência. O segundo é `Runtime Orchestration`, responsável por receber comandos de alto nível como start, stop, delete e traduzi-los em workflows. O terceiro é `Supervision`, onde residem processo pai do host, init interno, sinais, wait, reape e teardown. O quarto é `Observability`, responsável por eventos, métricas, logs e read models temporais.

A separação crítica é entre `Runtime Orchestration` e `Supervision`. O primeiro vive bem em Python. O segundo precisa tocar kernel semantics continuamente e deve viver em C++. Misturar ambos em um único módulo Python/Cython cria uma camada que é ao mesmo tempo negócio, transporte e syscalls, o que é a receita clássica para um acoplamento difícil de evoluir.

7.2 Agregados e entidades

O agregado principal do lado Python é `ContainerAggregate`. Ele encapsula `ContainerId`, `DesiredSpec`, `LifecyclePolicy`, `SignalPolicy`, `RestartPolicy` e uma projeção do estado observado. Já do lado do supervisor, a entidade operacional principal é `Task`, com um Ta

`skId`, um `ProcessRecord` do `init` e zero ou mais `ExecProcessRecords`. Essas entidades não precisam ter o mesmo shape nas duas linguagens. Elas precisam compartilhar apenas um contrato semântico.

Um erro comum em projetos mistos Python/C++ é tentar ter “a mesma classe” nos dois lados. O que deve ser estável é o contrato, não a representação. Python modela domínio e persistência. C++ modela execução e causalidade de processo.

7.3 Value objects

Este problema é rico em value objects. `SignalSpec`, `GracePeriod`, `ExitStatus`, `ExitTimestamp`, `CgroupPath`, `BundlePath`, `SupervisorEndpoint`, `NamespacePlan`, `ProcessScope` e `ShutdownPolicy` são todos bons candidatos. Eles tornam o código mais explícito e evitam uma infestação de inteiros, strings e booleans soltos, especialmente através da fronteira Cython.

No lado C++, value objects fortes também ajudam na leitura e no encapsulamento de invariantes. Um `GracePeriod` não é só um `uint64_t`; ele é um tipo que sabe se está desabilitado, qual sua duração monotônica e como virar um deadline de `timerfd`.

7.4 Domain services

`ShutdownCoordinator` é um domain service natural. Ele calcula a sequência de parada com base em política, estado e deadlines. `SignalPolicyResolver` resolve qual sinal inicial usar, se a policy exige group signaling e qual escalonamento aplicar. `SupervisorRecoveryService` roda no control plane e tenta reanexar ou limpar supervisores órfãos na subida do sistema. `ExitInfoProjector` transforma fatos do supervisor em um read model amigável para API e UI.

Esses serviços impedem que o código de parada se dissolva em vários lugares. Em runtimes jovens, a lógica de stop frequentemente começa na API, continua no worker, termina no wrapper shell e explode no runtime nativo. O resultado é impossível de testar. DDD, aqui, é menos uma questão de elegância teórica e mais uma ferramenta contra entropia.

7.5 Eventos de domínio

O modelo deve tratar fatos de lifecycle como eventos: `ContainerCreateRequested`, `SupervisorAttached`, `TaskStarted`, `StopRequested`, `GraceTimeoutExpired`, `ForceKillIssued`, `TaskExited`, `TaskReaped`, `ContainerResourcesDeleted`, `SupervisorReattached`, `SupervisorLost`. O supervisor nativo produz eventos operacionais; o control plane os transforma em eventos de domínio e atualiza projeções.

Essa separação é útil porque nem todo evento operacional merece ser persistido como evento de negócio. Mas toda decisão de negócio relevante precisa ter base em fatos operacionais reais. Eventos são a ponte.

8. Padrões de design aplicáveis

8.1 State

O padrão `State` é provavelmente o mais importante para esta feature. Um container não aceita os mesmos comandos em `creating`, `running`, `stopping` e `stopped`. Tentar codificar isso só com `if` espalhado invariavelmente gera buracos semânticos. O supervisor deve ter um state machine explícito, idealmente com objetos de estado ou uma tabela de transições bem definida. O control plane também pode usar `State`, mas com outra granularidade, para diferenciar `desired state` de `observed state`.

8.2 Strategy

`Strategy` encaixa perfeitamente em políticas de forwarding de sinais, escalonamento de `stop` e `recovery`. `DirectChildForwarding`, `ProcessGroupForwarding`, `GracefulThenCgroupKill`, `GracefulThenPidfdKill`, `FailStopOnSupervisorDeath` e `ContinueAndReattach` são estratégias reais que o runtime pode suportar. Isso evita “ifs de policy” enterrados em vários lugares e permite testar combinações sem duplicar o `core`.

8.3 Command

No lado Python, `start`, `stop`, `kill`, `wait`, `delete` e `exec` devem ser tratados como comandos de aplicação. Isso é útil para auditoria, `retry`, `idempotência` e fila de reconciliação. No lado nativo, o RPC local também é naturalmente `command-oriented`: cada request produz um efeito bem delimitado. O padrão `Command`, aqui, não é academicismo; é a forma mais limpa de manter trilhas operacionais e semântica `idempotente`.

8.4 Observer / Publisher-Subscriber

Eventos de lifecycle se encaixam naturalmente em `observer` ou `pub-sub`. O supervisor publica fatos como `started`, `exit`, `delete`. O control plane assina e atualiza seu `read model`. A documentação do `containerd` enfatiza a necessidade de um modelo assíncrono de eventos com ordem garantida exatamente para que camadas superiores como Docker recebam os fatos corretamente.[R10] Reproduzir essa ideia no projeto é prudente.

8.5 Facade e Adapter

O Cython layer deve funcionar como `Adapter`, escondendo detalhes de ABI nativa e expondo ao Python uma interface simples e idiomática. Já o launcher C++ ou a biblioteca cliente que Cython consome deve agir como `Facade` sobre o mundo mais complexo do supervisor local, `pidfds`, `sockets`, `FDs` e serialização. Essa dupla reduz drasticamente a superfície de acoplamento entre as linguagens.

8.6 Builder

A montagem do `ContainerSpec` nativo pede um `Builder`. Em Python, o domínio monta o spec de forma declarativa; Cython converte; C++ recebe uma estrutura já validada e pronta para execução. Um builder explícito ajuda a evitar “construtores com trinta parâmetros” e centraliza validação de invariantes como paths absolutos, timeout positivo, política de signal conhecida e compatibilidade entre namespaces e cgroups.

8.7 Template Method

O pipeline de lifecycle do supervisor pode ser modelado com `Template Method`: `prepare_bundle()`, `mount_rootfs()`, `start_init()`, `publish_started()`, `wait_or_subscribe()`, `graceful_stop()`, `force_kill_if_needed()`, `teardown()`. As diferenças entre drivers, futuros runtimes ou modos `rootless/rootful` entram em pontos de variação previsíveis. Isso é melhor do que clonar o pipeline inteiro para cada modo.

9. Requisitos funcionais

9.1 Requisitos centrais

RF-01. O sistema deve instanciar, para cada container, um supervisor nativo no host antes do `start` do processo do container. Esse supervisor deve permanecer vivo independentemente do processo Python que o criou e deve servir uma API local para `state`, `kill`, `wait`, `delete`, `shutdown` e `reattach`.

RF-02. O sistema deve injetar um `init` interno no bundle do container e executá-lo como `PID 1` no namespace de `PID` sempre que o modo supervisionado estiver habilitado. O `init` deve iniciar a workload real como filho direto e permanecer vivo até o término final do container.

RF-03. O supervisor deve manter o `pidfd` do processo `init` do container — ou um equivalente semanticamente seguro em plataformas futuras — e usá-lo como referência canônica para sinalização e observação de término sempre que suportado.[R13][R14][R15]

RF-04. O supervisor deve servir `wait` de maneira repetível. O primeiro término observado gera um `ExitInfo` imutável, persistido em memória do supervisor e, quando necessário, em estado serializado local. Chamadas posteriores de `wait` devem retornar esse mesmo resultado, e nunca depender de `reap` atrasado.

RF-05. O supervisor deve `reaped` imediatamente o `init` do container e quaisquer processos sob sua responsabilidade assim que o kernel reportar sua saída. O sistema não pode depender de chamadas externas de `wait` para evitar zombies.

RF-06. O `init` interno deve `reaped` filhos órfãos dentro do namespace do container e deve observar `SIGCHLD` continuamente até o término do container. Esse requisito é obrigatório mesmo que a workload declarada seja “single process”, porque shells, wrappers, exec helpers e libraries frequentemente criam descendentes invisíveis à API.

RF-07. O sistema deve suportar parada graciosa com sinal configurável e `grace period` configurável, alinhando-se conceitualmente a `STOPSIGNAL` e `stop_grace_period` do ecossistema Docker.[R3][R8]

RF-08. O sistema deve suportar escalonamento de parada para modo forçado. Em `cgroup v2`, a implementação recomendada é `cgroup.kill`; quando indisponível, deve existir estratégia de fallback explicitamente documentada.[R19]

RF-09. O supervisor deve ser dono canônico do `stdin`, `stdout`, `stderr` e/ou `PTY` do container, permitindo `reattach` de clientes e drenagem ordenada do I/O após o término do processo.[R11]

RF-10. O sistema deve publicar eventos de lifecycle em ordem causal mínima: `create` antes de `start`, `start` antes de `exit`, `exit` antes de `delete`. O modelo de eventos não pode permitir que `exit` seja observado antes de `start` ter sido confirmado ao chamador.[R10]

RF-11. O control plane deve ser capaz de reanexar-se a supervisores vivos durante sua própria recuperação, reconstruindo o estado observado dos containers sem reiniciar a workload.

RF-12. O sistema deve oferecer uma operação de `delete` segura que só finalize `tear-down` de recursos depois que o container estiver parado e o sub-tree de processos estiver vazio ou comprovadamente morto. O uso de `cgroup.events.populated` como gatilho de `cleanup` é fortemente recomendado.[R19]

RF-13. O sistema deve diferenciar semântica de “stop”, “kill” e “delete force”. Essas operações podem compartilhar partes do pipeline, mas não devem ser tratadas como sinônimos no modelo de domínio nem na API.

RF-14. O design deve reservar identidade separada para processos `exec` adicionais, ainda que o primeiro release não exponha toda a superfície de `exec`. Se a modelagem inicial não comportar isso, a extensão futura será muito mais cara.

9.2 Requisitos funcionais derivados

RF-15. O init interno deve suportar ao menos duas políticas de forwarding de sinal: para o filho direto e para o grupo de processos da workload. A escolha deve ser configurável por policy do container.

RF-16. O supervisor deve expor timestamps monotônicos e wall clock para eventos de término, permitindo medir latência de parada sem depender apenas de relógio do sistema.

RF-17. O supervisor deve permitir fechamento explícito de stdin, porque algumas aplicações dependem desse evento para sair de forma cooperativa.

RF-18. O sistema deve registrar de forma auditável qual sinal foi enviado, quando foi enviado, se houve timeout de grace period e qual mecanismo de força foi usado no final do pipeline.

10. Requisitos não funcionais

RNF-01 — Correção semântica. A prioridade número um é correção do lifecycle, não throughput. O runtime não pode deixar zombies, não pode confundir desired state com observed state e não pode perder o `exit status` do processo principal.

RNF-02 — Tolerância a reinício do control plane. Reiniciar a API Python, reiniciar workers ou recarregar código não deve matar nem dessupervisionar containers vivos. A arquitetura deve permitir reattach a supervisores persistentes.

RNF-03 — Determinismo temporal. Deadlines e grace periods devem usar relógio monotônico no lado nativo. O sistema não deve depender de `time.sleep()` em Python ou de relógio wall clock para decisões de escalonamento.

RNF-04 — Observabilidade. Devem existir logs estruturados e métricas como `graceful_stop_duration`, `force_kill_total`, `zombies_reaped_total`, `supervisor_reattach_total`, `unknown_state_total`, `wait_latency_ms` e `cleanup_latency_ms`.

RNF-05 — Segurança operacional. O supervisor e o init interno devem minimizar privilégios e superfícies de parsing. O init injetado deve ser pequeno, idealmente estático, com dependências mínimas. O host supervisor deve manipular apenas os FDs e capacidades estritamente necessários.

RNF-06 — Testabilidade. A máquina de estados do supervisor deve ser testável sem containers reais sempre que possível, por meio de doubles de eventos e relógio. Testes de integração com o kernel devem existir para os cenários críticos, mas o coração da lógica não pode estar dissolvido em código impossível de simular.

RNF-07 — Compatibilidade explícita. O runtime deve declarar de maneira franca seu baseline: Linux moderno, `cgroup v2`, `pidfd`, `signalfd` e namespaces relevantes. Compatibilidade com kernels antigos deve ser tratada como feature separada, não como dívida escondida.

RNF-08 — Estabilidade da fronteira Python/C++. O contrato entre Cython e C++ deve ser estreito e estável. O ideal é um client API pequeno, versionado e com ownership de memória claramente documentado.

RNF-09 — Overhead previsível. O custo de um processo supervisor e um init interno por container deve ser aceito e medido. Otimizações 1:N só devem entrar quando a semântica 1:1 estiver provada.

11. Cython como cola: o que fazer e o que evitar

A documentação do Cython deixa três mensagens muito úteis para este projeto. Primeiro, ele oferece wrapping direto de C++ por meio de `cdef extern from` e `cppclass`, incluindo construtores e métodos com tradução de exceções `except +`. [R23][R25] Segundo, ele permite liberar o GIL em seções que chamam código externo bloqueante, o que é exatamente o que se deseja em chamadas locais ao supervisor ou esperas de I/O. [R24] Terceiro, ele documenta cuidados de freethreading e callbacks entre C/C++ e Python, reforçando que callbacks nativos devem reacquirir o thread-state com disciplina e que versões `py_safe` de primitivas de sincronização existem para alguns casos. [R26]

A partir disso, a recomendação prática é simples. Use Cython para marshaling de requests/responses, inicialização do launcher, conexão com o socket local do supervisor e eventual consumo de eventos em uma thread dedicada que reacquire o GIL apenas no momento de entregar fatos ao Python. Não use Cython para deixar o C++ chamar diretamente objetos Python a partir do thread de reaping ou de handlers de sinal. Esse caminho é tecnicamente possível, mas operacionalmente frágil.

Também recomendo fortemente que o Cython não envolva classes C++ internas e profundas do runtime. Em vez disso, o C++ deve expor uma fachada estreita, preferencialmente com uma ABI simples, por exemplo:

```
cdef extern from "runtime_client.h" nogil:
    cdef struct RtCreateRequest:
        pass
    cdef struct RtCreateResponse:
        pass
    int rt_create(RtCreateRequest* req, RtCreateResponse* resp) noexcept
    int rt_wait(const char* id, int timeout_ms, RtCreateResponse* resp) noexcept
```

Essa fronteira é mais estável do que um grande wrapping de objetos C++ ricos. Cython, aqui, é uma ferramenta de adaptação e desempenho, não o repositório de verdade do domínio de runtime.

12. Plano de implementação recomendado

Fase 1 — Provar o modelo sem namespaces

A primeira fase deve ignorar containers por um momento e provar apenas o modelo de supervisão. Implemente um supervisor C++ que cria um processo filho comum, mantém `pidfd`, faz `waitid(P_PIDFD)`, usa `signalfd`, gerencia `grace period` com `timerfd` e publica um `ExitInfo` cacheado. Em paralelo, implemente um `init` interno mínimo que apenas cria um filho, repassa sinais e `reaped` órfãos. Mesmo sem namespaces, isso já valida `kill`, `wait`, `reap` e `shutdown` limpo.

Essa fase é crucial porque isola a semântica de processo do resto da complexidade do runtime. Se você não consegue acertar isso fora de namespaces, também não acertará dentro deles.

Fase 2 — Integrar namespaces, rootfs e cgroup v2

Na segunda fase, integre o pipeline real de criação do container. O supervisor passa a preparar `bundle`, montar `rootfs`, configurar `cgroup` e iniciar o `init` interno como `PID 1`. Introduza `cgroup.events` e `cgroup.kill` no fluxo de `cleanup`.^[R19] Nesta fase, a operação `delete` já deve ser capaz de `teardown`ear tudo depois do término do processo.

A recomendação é não introduzir `exec` ainda. Primeiro prove `create/start/stop/wait/delete` com I/O reanexável.

Fase 3 — Integrar o control plane Python

Só depois de o runtime nativo estar semanticamente correto deve entrar a integração com Python. O control plane passa a persistir `desired state`, lançar supervisores via Cython e reanexar-se a eles após reinício. Nesta fase, a API externa já pode oferecer `run`, `ps`, `logs`, `stop`, `kill`, `wait` e `rm` com comportamento confiável.

É nesta fase que a fronteira Cython precisa ser mantida pequena. Quanto mais simples for o contrato, menor a chance de o control plane contaminar a semântica do supervisor.

Fase 4 — Ordenação de eventos e observabilidade forte

Introduza event stream local, read models e métricas. Prove que `start` nunca é observado depois de `exit`, que `wait` depois do término é imediato e que `reattach` reconstrói a visão correta. Implemente tracing interno do pipeline de stop, incluindo envio de sinal, expiração do grace period, escalonamento e término efetivo.

Fase 5 — `exec`, corridas e recuperação pessimista

Com a base pronta, entre nos casos difíceis: `exec` concorrente, saída do `init` ao mesmo tempo que um processo `exec`, perda de conexão com supervisor, restart do daemon durante parada, órfãos de `setns`, I/O ainda drenando no momento do `exit` e limpeza tardia do rootfs. Aqui valem muito os testes que reproduzem os tipos de corrida observados no ecossistema containerd, porque eles mostram exatamente onde o modelo aparentemente “quase certo” quebra.

Fase 6 — Hardening e políticas avançadas

A fase final introduz políticas avançadas: different signal forwarding strategies, `PDEATHSIG` opcional, rootless roadmap, supervisão 1:N quando fizer sentido, integração com health checks, restart policies mais sofisticadas e modos específicos para workloads que exigem ser `PID 1` reais.

13. Estratégia de testes e cenários obrigatórios

Um runtime desse tipo só é confiável se sobreviver a testes maldosos. O mínimo aceitável inclui um processo que ignora `SIGTERM`, um wrapper shell que não encaminha sinais, um filho que faz double-fork, uma workload que termina instantaneamente antes do `start` retornar, uma parada em que `stdout` ainda está sendo escrito no momento do `exit`, uma expiração de grace period seguida de `cgroup.kill`, um reinício do control plane durante `running` e outro durante `stopping`, e um cenário de `exec` em que um helper gera órfãos no namespace do container.

Também é essencial medir ausência de zombies. O sucesso não é apenas “o container saiu com o código certo”; é “o container saiu com o código certo e nenhuma entrada de processo foi deixada para trás”. Outro conjunto de testes importante envolve idempotência: chamar `stop` duas vezes, `wait` antes e depois do término, `delete` depois de `stopped`, `kill` em `stopped`, e `reattach` múltiplo ao mesmo supervisor.

Por fim, recomendo testes de ordem causal de eventos. Eles são o tipo de verificação que ninguém escreve no protótipo e que depois cobra juros altos. Prove por teste que `TaskStart` sempre antecede `TaskExit` na visão do control plane. O containerd explicita esse requisito por uma razão.[R10]

14. Conclusão

A implementação correta do modelo “processo pai do host + init interno; kill, wait, reap, shutdown limpo” exige abandonar a ideia de que o problema é apenas “criar um processo num namespace” e aceitar que o verdadeiro produto aqui é um contrato de supervisão. Esse contrato tem semântica temporal, semântica de ownership de FDs, semântica de causalidade de eventos e semântica de limpeza. Ele é o ponto em que o runtime deixa de ser uma sequência de syscalls e vira um sistema confiável.

No contexto do projeto proposto, a forma mais sólida de alcançar esse objetivo é manter Python como control plane e colocar a autoridade operacional em um supervisor C++ externo e reanexável, assistido por um init interno C++ pequeno e estático dentro do container. Cython, por sua vez, deve permanecer como fronteira de adaptação eficiente, liberando o GIL onde importa e traduzindo contratos, não como o local em que a árvore de processos realmente vive.

Se eu tivesse de escolher apenas três decisões como “não negociáveis” para o primeiro release, seriam estas. A primeira é um supervisor nativo por container, fora do processo Python. A segunda é um init interno padrão, não opcional, para resolver PID 1, forwarding e reaping. A terceira é assumir Linux moderno com `pidfd`, `waitid(P_PIDFD)`, `signalfd` e `cgroup v2`, em vez de diluir o design em fallbacks antigos logo no início. Com essas três decisões, o projeto ganha uma base coerente sobre a qual `exec`, restart policies, health checks, snapshots e outras features poderão ser construídas sem reabrir a fundação.

Referências

[R1] Docker Docs. *Running containers*. Disponível em: <https://docs.docker.com/engine/containers/run/> . Acesso em: 12 mar. 2026.

[R2] Docker Docs. *dockerd*. Disponível em: <https://docs.docker.com/reference/cli/dockerd/> . Acesso em: 12 mar. 2026.

[R3] Docker Docs. *docker container stop*. Disponível em: <https://docs.docker.com/reference/cli/docker/container/stop/> . Acesso em: 12 mar. 2026.

[R4] Docker Docs. *docker container attach*. Disponível em: <https://docs.docker.com/reference/cli/docker/container/attach/> . Acesso em: 12 mar. 2026.

[R5] Docker Docs. *JSONArgsRecommended*. Disponível em: <https://docs.docker.com/reference/build-checks/json-args-recommended/> . Acesso em: 12 mar. 2026.

[R6] Docker Docs. *Building best practices*. Disponível em: <https://docs.docker.com/build/building/best-practices/> . Acesso em: 12 mar. 2026.

[R7] Docker Docs. *Run multiple processes in a container*. Disponível em: https://docs.docker.com/engine/containers/multi-service_container/ . Acesso em: 12 mar. 2026.

[R8] Docker Docs. *Compose file reference — services*. Disponível em: <https://docs.docker.com/reference/compose-file/services/> . Acesso em: 12 mar. 2026.

[R9] Open Container Initiative. *OCI Runtime Spec — runtime.md*. Disponível em: <https://github.com/opencontainers/runtime-spec/blob/main/runtime.md> . Acesso em: 12 mar. 2026.

[R10] containerd. *Runtime v2 README*. Disponível em: <https://github.com/containerd/containerd/blob/main/core/runtime/v2/README.md> . Acesso em: 12 mar. 2026.

[R11] containerd. *runtime/task/v2/shim.proto*. Disponível em: <https://github.com/containerd/containerd/blob/main/api/runtime/task/v2/shim.proto> . Acesso em: 12 mar. 2026.

[R12] man7.org. *PR_SET_CHILD_SUBREAPER(2const)*. Disponível em: https://man7.org/linux/man-pages/man2/pr_set_child_subreaper.2const.html . Acesso em: 12 mar. 2026.

[R13] man7.org. *pidfd_open(2)*. Disponível em: https://man7.org/linux/man-pages/man2/pidfd_open.2.html . Acesso em: 12 mar. 2026.

- [R14]** man7.org. *pidfd_send_signal(2)*. Disponível em: https://man7.org/linux/man-pages/man2/pidfd_send_signal.2.html . Acesso em: 12 mar. 2026.
- [R15]** man7.org. *wait(2)*. Disponível em: <https://man7.org/linux/man-pages/man2/wait.2.html> . Acesso em: 12 mar. 2026.
- [R16]** man7.org. *signalfd(2)*. Disponível em: <https://man7.org/linux/man-pages/man2/signalfd.2.html> . Acesso em: 12 mar. 2026.
- [R17]** man7.org. *PR_SET_PDEATHSIG(2const)*. Disponível em: https://man7.org/linux/man-pages/man2/pr_set_pdeathsig.2const.html . Acesso em: 12 mar. 2026.
- [R18]** man7.org. *pid_namespaces(7)*. Disponível em: https://man7.org/linux/man-pages/man7/pid_namespaces.7.html . Acesso em: 12 mar. 2026.
- [R19]** Linux Kernel Documentation. *Control Group v2*. Disponível em: <https://www.kernel.org/doc/html/v6.1/admin-guide/cgroup-v2.html> . Acesso em: 12 mar. 2026.
- [R20]** krallin/tini. *README*. Disponível em: <https://github.com/krallin/tini/blob/master/README.md> . Acesso em: 12 mar. 2026.
- [R21]** Yelp. *dumb-init*. Disponível em: <https://github.com/Yelp/dumb-init> . Acesso em: 12 mar. 2026.
- [R22]** Python Software Foundation. *What's New in Python 3.9; signal module; What's New in Python 3.12*. Disponível em: <https://docs.python.org/3/whatsnew/3.9.html> ; <https://docs.python.org/3/library/signal.html> ; <https://docs.python.org/3/whatsnew/3.12.html> . Acesso em: 12 mar. 2026.
- [R23]** Cython Documentation. *Using C++ in Cython*. Disponível em: https://cython.readthedocs.io/en/latest/src/userguide/wrapping_CPlusPlus.html . Acesso em: 12 mar. 2026.
- [R24]** Cython Documentation. *Interfacing with External C Code*. Disponível em: https://cython.readthedocs.io/en/latest/src/userguide/external_C_code.html . Acesso em: 12 mar. 2026.
- [R25]** Cython Documentation. *Language Basics*. Disponível em: https://cython.readthedocs.io/en/latest/src/userguide/language_basics.html . Acesso em: 12 mar. 2026.
- [R26]** Cython Documentation. *Free threading*. Disponível em: <https://cython.readthedocs.io/en/latest/src/userguide/freethreading.html> . Acesso em: 12 mar. 2026.

[R27] Docker Docs. *docker container kill*. Disponível em: <https://docs.docker.com/reference/cli/docker/container/kill/> . Acesso em: 12 mar. 2026.